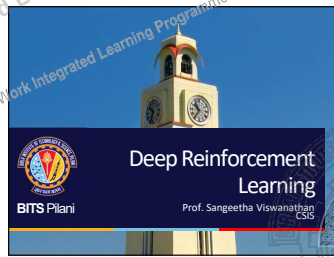
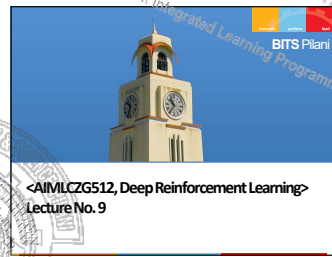


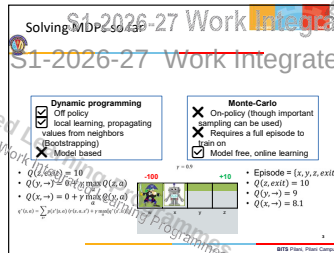
8/17/2026



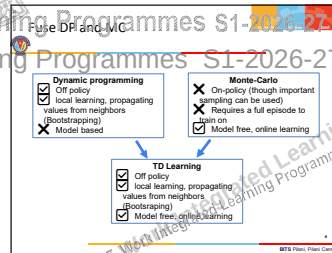
1



2



3



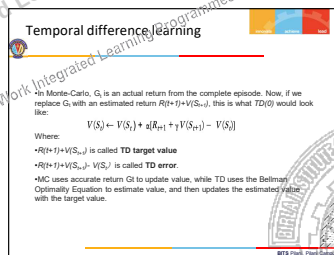
4

BITS WILP | Generated: 17-Aug-2026 12:13:13

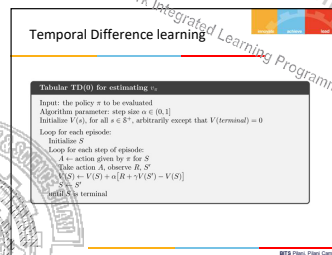
1

BITS Pilani, Pilani Campus

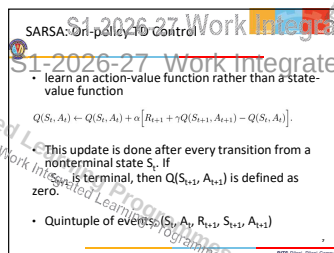
8/17/2026



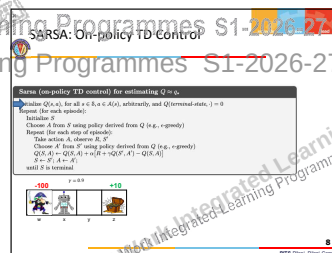
5



6



7



8

BITS WILP | Generated: 17-Aug-2026 12:13:13

2

BITS Pilani, Pilani Campus

SARSA: On-policy TD Control

SARSA: On-policy TD Control

Initialize $Q(s, a)$, for all $s \in S, a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Sarsa (on-policy TD control) for estimating $Q \approx q_*$.


 $Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$

Q =

8	6.8	8.8	9
w	x	y	z

10

GTS Planet, Planet Campus

SARSA: On-policy TD Control

SARSA: On-policy TD Control

Sarsa (on-policy TD control) for estimating $Q \approx q_*$.

Sarsa (on-policy TD control) for estimating $Q \approx q_*$.



 $Q = \begin{bmatrix} 5 & 6.9 & 6.0 & 9 \\ w & x & y & z \end{bmatrix}$

SARSA: On-policy TD Control

Initial: $Q(s, a)$, for all $s \in S$, $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Sarsa (on-policy TD control) for estimating $Q \approx q_*$.

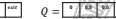
-100 **+10**


w x y z

w	x	y	w	x
0	1	0	0	1

$Q =$

0	0	0	0
w	x	y	z



13

©2015 Shady Side Academy

Figure 10 shows a 4-state MDP with states w, x, y, and z. The actions available are 'wait' and 'exit'. The transition probabilities are given in the table below. The value function V is shown for each state, with the optimal action highlighted in red. The optimal policy is: w to x, x to x, y to x, and z to exit.

	wait	-100	exit	wait
w	0	0	0	0
x	0	0	0	0
y	0	0	0	0
z	0	0	0	0

SARSA: On-policy TD Control

SARSA: On-policy TD-Contr

Sarsa (on-policy TD control) for estimating $Q \approx q_*$.

Sarsa (on-policy TD control) for estimating $Q \approx q_*$.









W	X	Y	Z
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

W	X	Y	Z
17	18	19	20
21	22	23	24
25	26	27	28
29	30	31	32

8/17/2026

SARSA: On-policy TD Control

SARSA (on-policy TD control) for estimating $Q \approx q_*$.

Initialize $Q(s,a)$ for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, *) = 0$.

Repeat (for each episode):

Initialize S .

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Repeat (for each step of episode):

Take action A , observe R, S' .

Choose A' from π using policy derived from Q (e.g., ϵ -greedy).

$\delta = R + \gamma V(S') - V(S)$.

$Q(S,A) \leftarrow Q(S,A) + \alpha \delta$.

Set $S = S'$.

until S is terminal.

$\gamma = 0.5$

$Q = \begin{bmatrix} 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \end{bmatrix}$

17

SARSA: On-policy TD Control

SARSA (on-policy TD control) for estimating $Q \approx q_*$.

Initialize $Q(s,a)$ for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, *) = 0$.

Repeat (for each episode):

Initialize S .

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Repeat (for each step of episode):

Take action A , observe R, S' .

Choose A' from π using policy derived from Q (e.g., ϵ -greedy).

$\delta = R + \gamma V(S') - V(S)$.

$Q(S,A) \leftarrow Q(S,A) + \alpha \delta$.

Set $S = S'$.

until S is terminal.

$\gamma = 0.5$

$Q = \begin{bmatrix} 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \end{bmatrix}$

18

SARSA: On-policy TD control

SARSA (on-policy TD control) for estimating $Q \approx q_*$.

Initialize $Q(s,a)$ for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, *) = 0$.

Repeat (for each episode):

Initialize S .

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Repeat (for each step of episode):

Take action A , observe R, S' .

Choose A' from π using policy derived from Q (e.g., ϵ -greedy).

$\delta = R + \gamma V(S') - V(S)$.

$Q(S,A) \leftarrow Q(S,A) + \alpha \delta$.

Set $S = S'$.

until S is terminal.

$\gamma = 0.5$

$Q = \begin{bmatrix} 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \end{bmatrix}$

19

Q-learning: Off-policy TD Control

Use the original TD update rule

- $Q(s,a) = Q(s,a) + \alpha (R_{t+1} + \gamma \max_{a'} Q(s',a') - Q(s,a))$
- Approximates the state-action value for the optimal policy, i.e., q_*
 - Assuming that every state-action pair is visited infinitely often
- Follows from the proof of convergence for the Bellman function
 - See slides MDPs+DP

20

BITS WILP | Generated: 17-Aug-2026 12:13:13

5

BITS Pilani, Pilani Campus

8/17/2026

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating $q \approx q_*$.

Initialize $Q(s,a)$ for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, *) = 0$.

Repeat (for each episode):

Initialize S .

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$\delta = R + \gamma \max_{a'} Q(S',a') - Q(S,A)$.

$Q(S,A) \leftarrow Q(S,A) + \alpha \delta$.

Set $S = S'$.

until S is terminal.

$\gamma = 0.5$

$Q = \begin{bmatrix} 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \end{bmatrix}$

21

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating $q \approx q_*$.

Initialize $Q(s,a)$ for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, *) = 0$.

Repeat (for each episode):

Initialize S .

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$\delta = R + \gamma \max_{a'} Q(S',a') - Q(S,A)$.

$Q(S,A) \leftarrow Q(S,A) + \alpha \delta$.

Set $S = S'$.

until S is terminal.

$\gamma = 0.5$

$Q = \begin{bmatrix} 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \end{bmatrix}$

22

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating $q \approx q_*$.

Initialize $Q(s,a)$ for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, *) = 0$.

Repeat (for each episode):

Initialize S .

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$\delta = R + \gamma \max_{a'} Q(S',a') - Q(S,A)$.

$Q(S,A) \leftarrow Q(S,A) + \alpha \delta$.

Set $S = S'$.

until S is terminal.

$\gamma = 0.5$

$Q = \begin{bmatrix} 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \end{bmatrix}$

23

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating $q \approx q_*$.

Initialize $Q(s,a)$ for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, *) = 0$.

Repeat (for each episode):

Initialize S .

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$\delta = R + \gamma \max_{a'} Q(S',a') - Q(S,A)$.

$Q(S,A) \leftarrow Q(S,A) + \alpha \delta$.

Set $S = S'$.

until S is terminal.

$\gamma = 0.5$

$Q = \begin{bmatrix} 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \\ 100 & 60 & 50 & 0 \end{bmatrix}$

24

BITS WILP | Generated: 17-Aug-2026 12:13:13

6

BITS Pilani, Pilani Campus

8/17/2026

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating $v \approx \pi_*$.

Initialize $Q(s,a)$, for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Repeat (for each episode):

Initialize S .

Repeat (for each step of episode):

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$.

$S \leftarrow S'$.

until S is terminal.

$\gamma = 0.9$

$+10$ -100

$Q = \begin{bmatrix} 1 & 0.6 & 0.9 & 1 \end{bmatrix}$

25

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating $v \approx \pi_*$.

Initialize $Q(s,a)$, for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Repeat (for each episode):

Initialize S .

Repeat (for each step of episode):

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$.

$S \leftarrow S'$.

until S is terminal.

$\gamma = 0.9$

$+10$ -100

$Q = \begin{bmatrix} 1 & 0.6 & 0.9 & 1 \end{bmatrix}$

26

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating $v \approx \pi_*$.

Initialize $Q(s,a)$, for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Repeat (for each episode):

Initialize S .

Repeat (for each step of episode):

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$.

$S \leftarrow S'$.

until S is terminal.

$\gamma = 0.9$

$+10$ -100

$Q = \begin{bmatrix} 1 & 0.6 & 0.9 & 1 \end{bmatrix}$

27

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating $v \approx \pi_*$.

Initialize $Q(s,a)$, for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Repeat (for each episode):

Initialize S .

Repeat (for each step of episode):

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$.

$S \leftarrow S'$.

until S is terminal.

$\gamma = 0.9$

$+10$ -100

$Q = \begin{bmatrix} 1 & 0.6 & 0.9 & 1 \end{bmatrix}$

28

BITS WILP | Generated: 17-Aug-2026 12:13:13

7

BITS Pilani, Pilani Campus

8/17/2026

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating $v \approx \pi_*$.

Initialize $Q(s,a)$, for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Repeat (for each episode):

Initialize S .

Repeat (for each step of episode):

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$.

$S \leftarrow S'$.

until S is terminal.

$\gamma = 0.9$

$+10$ -100

$Q = \begin{bmatrix} 1 & 0.6 & 0.9 & 1 \end{bmatrix}$

29

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating $v \approx \pi_*$.

Initialize $Q(s,a)$, for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Repeat (for each episode):

Initialize S .

Repeat (for each step of episode):

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$.

$S \leftarrow S'$.

until S is terminal.

$\gamma = 0.9$

$+10$ -100

$Q = \begin{bmatrix} 1 & 0.6 & 0.9 & 1 \end{bmatrix}$

30

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating $v \approx \pi_*$.

Initialize $Q(s,a)$, for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Repeat (for each episode):

Initialize S .

Repeat (for each step of episode):

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$.

$S \leftarrow S'$.

until S is terminal.

$\gamma = 0.9$

$+10$ -100

$Q = \begin{bmatrix} 1 & 0.6 & 0.9 & 1 \end{bmatrix}$

31

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating $v \approx \pi_*$.

Initialize $Q(s,a)$, for all $s \in S$ and $a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Repeat (for each episode):

Initialize S .

Repeat (for each step of episode):

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$.

$S \leftarrow S'$.

until S is terminal.

$\gamma = 0.9$

$+10$ -100

$Q = \begin{bmatrix} 1 & 0.6 & 0.9 & 1 \end{bmatrix}$

32

BITS WILP | Generated: 17-Aug-2026 12:13:13

8

BITS Pilani, Pilani Campus

8/17/2026

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating q^{π} .

Initialize $Q(s,a)$, for all $s \in S, a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Repeat (for each episode):

Initialize S .

Repeat (for each step of episode):

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$.

$S \leftarrow S'$.

until S is terminal.

$Q = \begin{bmatrix} 7 & 82 & 98 & 100 \\ 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \end{bmatrix}$

33

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating q^{π} .

Initialize $Q(s,a)$, for all $s \in S, a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Repeat (for each episode):

Initialize S .

Repeat (for each step of episode):

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$.

$S \leftarrow S'$.

until S is terminal.

$Q = \begin{bmatrix} 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \end{bmatrix}$

34

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating q^{π} .

Initialize $Q(s,a)$, for all $s \in S, a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Repeat (for each episode):

Initialize S .

Repeat (for each step of episode):

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$.

$S \leftarrow S'$.

until S is terminal.

$Q = \begin{bmatrix} 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \end{bmatrix}$

35

Q-learning: Off-policy TD Control

Q-learning (off-policy TD control) for estimating q^{π} .

Initialize $Q(s,a)$, for all $s \in S, a \in A(s)$, arbitrarily, and $Q(\text{terminal state}, \cdot) = 0$.

Repeat (for each episode):

Initialize S .

Repeat (for each step of episode):

Choose A from π using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S' .

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$.

$S \leftarrow S'$.

until S is terminal.

And so on...

$Q = \begin{bmatrix} 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \end{bmatrix}$

36

BITS WILP | Generated: 17-Aug-2026 12:13:13

9

BITS Pilani, Pilani Campus

8/17/2026

Expected SARSA

Instead of the maximum over next state-action pairs it uses the $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma E_{\pi} Q(S_{t+1}, A_{t+1}) | S_t, A_t] - Q(S_t, A_t)$. account now likely each action is under the current policy

$Q = \begin{bmatrix} 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \end{bmatrix}$

37

Maximization Bias & Double Learning

- Maximum over estimated values is used implicitly as an estimate of the maximum value, which can lead to a significant positive bias.
- Solution – Learn two estimates – Double Learning
- Double learning doubles the memory requirements, but does not increase the amount of computation per step.

$Q = \begin{bmatrix} 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \end{bmatrix}$

38

n-step TD Prediction

One-step return: $G_{t+1} = R_{t+1} + \gamma V_t(S_{t+1})$

Two-step return: $G_{t+2} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V_t(S_{t+2})$

Ref Section 7.1 of TB

$Q = \begin{bmatrix} 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \end{bmatrix}$

39

n-step TD Prediction

One-step return: $G_{t+1} = R_{t+1} + \gamma V_t(S_{t+1})$

Two-step return: $G_{t+2} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V_t(S_{t+2})$

n-step return: $G_{t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V_t(S_{t+n})$

Ref Section 7.1 of TB

$Q = \begin{bmatrix} 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \\ 4 & 6 & 6 & 100 \end{bmatrix}$

40

BITS WILP | Generated: 17-Aug-2026 12:13:13

10

BITS Pilani, Pilani Campus

n-step TD Prediction

Integrated Learning Program

Prerequisites: Linear Algebra, Probability & Statistics, Reinforcement Learning

On-policy learning

$G_{t+k} = R_{t+k} + \gamma V(S_{t+k})$

Two-tuple return:

$$G_{t+2:t} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+3}(S_{t+2})$$

n-step return:

$$G_{t+k:t} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{k-1} R_{t+k} + \gamma^k V_{t+k+1}(S_{t+k})$$

State-value learning algorithm for using n-step returns:

$$V_{t+1}(S_t) = V_{t+1}(S_t) + \alpha [G_{t+2:t} - V_{t+1}(S_t)], \quad 0 \leq t < T$$

MAT Section 7.6 of RL

BBC's Press, Pressing

n-step TD Prediction

Background Learning Program

n-step TD for estimating v_{π}

```

Initialize a policy  $\pi$ 
Initialize parameters: step size  $\alpha \in (0, 1)$ ,  $\gamma$ , a positive integer  $n$ 
Initialize  $V(0)$  arbitrarily, for  $s \in \mathcal{S}$ 
At each state  $s$ , observe reward  $r_t$  and  $S_t$ , and  $S_{t+n}$  (so that both indices add  $n+1$ )
Loop for each episode
  Initialize  $s = S_t$ ,  $v = V(s)$ 
  for  $k = 0, 1, 2, \dots$ 
     $a_k \leftarrow \pi(s_k)$ 
    Take an action according to  $\pi(s_k)$ 
    Observe  $r_{k+1}$  and the next state  $s_{k+1}$ 
    if  $k \geq n-1$ , then  $V(s_{k+1}) \leftarrow r_{k+1} + \gamma V(s_{k+1})$ 
    if  $k \geq n-1$  or the state  $s_{k+1}$  is being updatd)
       $v \leftarrow (1-\alpha)v + \alpha(r_{k+1} + \gamma v - V(s_k))$ 
     $s \leftarrow s_{k+1}$ 
  end for
end for
  
```

8.785, Notes, Page 20